ENGR010 - S23 - PY 1-3

January 26, 2023

#

Weekly Python Assignment 1 - Group 3 - S23

ENGR010SP23PY01GR03

Instructions: For each exercise below, type the requested code into the cell below the exercise, then run your code to verify that it works correctly. If you don't see results when you run the cell, add a line to your code with a print statement and run the cell again.

```
<div style="float:left;width:35%;">
    <b>Color Code (nonfunctional in Google Colab):</b><br>
    - <mark style="color:purple;background:none">External File/Imported Module</mark><br>
    - <mark style="color:#EEC83A;background:none">User-created Function</mark><br>
    - <mark style="color:green;background:none">Python/Imported Function</mark><br>
    - <mark style="color:#EE6B3A;background:none">Name-specific Data Structure/Variable</mark><
</div>
<div style="float:left;width:65%;">
    <b>Relative Difficulty & Complexity</b><br>
    - <b>Easy (E):</b> Covered during the week's lecture. Typically single-step exercise<br>
    - <b>Medium (M):</b> Some combination with topics previously covered. Mostly double-step ex
    - <b>Hard (H):</b> Combination with topics previously covered. Multi-step exercise<br>
</div>

<br><b>Breakdown:</b> 1 (E), 5 (M), & 1 (H)
```

Example 1:ENGR010SP23PY01GR03EP01 Use Python as a calculator to find the difference between 23 and 14.

[1]:
```
23 - 14   #  Output shows up when you run the cell.
```

[1]:
```
9
```

Example 2:ENGR010SP23PY01GR03EP02 Set the variable named xry equal to 25. Print out the value of xry.

[2]:
```
xyz = 25   # When you run this cell, nothing gets printed unless you add the␣
 ↪print statement
print(xyz)
```

```
25
```

1(M).ENGR010SP23PY01GR03EX01 Use Python as a calculator to create the string "dumbdumbdumb" from the string "dumb". First, use string addition. Then do the same thing using string multiplication.

[3]:
```
"dumb" + "dumb" + "dumb"
```

[3]:
```
'dumbdumbdumb'
```

[4]:
```
"dumb" * 3
```

[4]:
```
'dumbdumbdumb'
```

2(M).ENGR010SP23PY01GR03EX02 Assign the sum of the numbers 1, 2, 3 to the variable sum. Explain why sum is a bad name for a custom variable in the markdown cell below.

[5]:
```
sum = 1 + 2 + 3
sum
```

[5]:
```
6
```

Why is this a bad name for a variable? (insert your answer here) This is because the python provides the command 'sum' by default. If programmar uses 'sum' for custom variable, the 'sum' command loses its existing function and is designated as a variable.

3(E).ENGR010SP23PY01GR03EX03 Use the integer 5 to create the string "55555".

Hint: Do not manually convert the integer to a string, use a Python function to do so.

[6]:
```
str(5)*5
```

[6]:
```
'55555'
```

4(M).ENGR010SP23PY01GR03EX04 Print the multiplication table for the integers 1 through 12. All numbers should be right-justified with a field width of 4.

[7]:
```
for i in range (1, 13):
    for j in range (1, 13):
        print(str(i).rjust(4) + "*" + str(j).rjust(4) + "=" + str(i*j).rjust(4))
```

```
   1*   1=   1
   1*   2=   2
   1*   3=   3
   1*   4=   4
   1*   5=   5
   1*   6=   6
   1*   7=   7
   1*   8=   8
   1*   9=   9
   1*  10=  10
   1*  11=  11
```

```
1*  12=  12
2*   1=   2
2*   2=   4
2*   3=   6
2*   4=   8
2*   5=  10
2*   6=  12
2*   7=  14
2*   8=  16
2*   9=  18
2*  10=  20
2*  11=  22
2*  12=  24
3*   1=   3
3*   2=   6
3*   3=   9
3*   4=  12
3*   5=  15
3*   6=  18
3*   7=  21
3*   8=  24
3*   9=  27
3*  10=  30
3*  11=  33
3*  12=  36
4*   1=   4
4*   2=   8
4*   3=  12
4*   4=  16
4*   5=  20
4*   6=  24
4*   7=  28
4*   8=  32
4*   9=  36
4*  10=  40
4*  11=  44
4*  12=  48
5*   1=   5
5*   2=  10
5*   3=  15
5*   4=  20
5*   5=  25
5*   6=  30
5*   7=  35
5*   8=  40
5*   9=  45
5*  10=  50
5*  11=  55
```

5* 12= 60
6* 1= 6
6* 2= 12
6* 3= 18
6* 4= 24
6* 5= 30
6* 6= 36
6* 7= 42
6* 8= 48
6* 9= 54
6* 10= 60
6* 11= 66
6* 12= 72
7* 1= 7
7* 2= 14
7* 3= 21
7* 4= 28
7* 5= 35
7* 6= 42
7* 7= 49
7* 8= 56
7* 9= 63
7* 10= 70
7* 11= 77
7* 12= 84
8* 1= 8
8* 2= 16
8* 3= 24
8* 4= 32
8* 5= 40
8* 6= 48
8* 7= 56
8* 8= 64
8* 9= 72
8* 10= 80
8* 11= 88
8* 12= 96
9* 1= 9
9* 2= 18
9* 3= 27
9* 4= 36
9* 5= 45
9* 6= 54
9* 7= 63
9* 8= 72
9* 9= 81
9* 10= 90
9* 11= 99

```
 9*  12= 108
10*   1=  10
10*   2=  20
10*   3=  30
10*   4=  40
10*   5=  50
10*   6=  60
10*   7=  70
10*   8=  80
10*   9=  90
10*  10= 100
10*  11= 110
10*  12= 120
11*   1=  11
11*   2=  22
11*   3=  33
11*   4=  44
11*   5=  55
11*   6=  66
11*   7=  77
11*   8=  88
11*   9=  99
11*  10= 110
11*  11= 121
11*  12= 132
12*   1=  12
12*   2=  24
12*   3=  36
12*   4=  48
12*   5=  60
12*   6=  72
12*   7=  84
12*   8=  96
12*   9= 108
12*  10= 120
12*  11= 132
12*  12= 144
```

5(H).ENGR010SP23PY01GR03EX05 Use complex multiplication to show that the square root of 2j is 1+j and the square root of -2j is 1-j.

[8]:
```
import math, sys

var1 = 2j**(1/2)
print(var1)
var2 = (-2j)**(1/2)
print(var2)
#Python floating point representation limits
```

```
#Machine Epsilon can be used to solve the floating point problem and prove that it is the same.
print("square root of 2j is ({0} + {1}j)".format(var1.real - sys.float_info.epsilon, var1.imag - sys.float_info.epsilon))
print("square root of -2j is ({0} + {1}j)".format(var2.real - sys.float_info.epsilon, var2.imag + sys.float_info.epsilon))
```

```
(1.0000000000000002+1.0000000000000002j)
(1.0000000000000002-1.0000000000000002j)
square root of 2j is (1.0 + 1.0j)
square root of -2j is (1.0 + -1.0j)
```

6(M).ENGR010SP23PY01GR03EX06 Using arbitrary precision floating point numbers, calculate the cube root of 5 to 75 decimal places.

[15]:
```
var3 = (5**(1/3))
print(f"{var3:.75}")
```

```
1.709975946676696834103381661407183855772018432617187 5
```

[16]:
```
from decimal import *
var3 = Decimal(5**(1/3))
getcontext().prec = 75
print(var3)
```

```
1.709975946676696834103381661407183855772018432617187 5
```

7(M).ENGR010SP23PY01GR03EX07 If an initial amount A is invested for n years at an annual return of p percent, it will grow to A(1 + p/100)n. Create a table showing the yearly growth of A over a 20-year period. Right justify all values and be sure to create column headings indicating the years and amounts. Use initial amount A of 200 and annual return p of 6.

[10]:
```
for n in range (1,21):
    print("year:" + "{0:>2}".format(n), end = " ")
    print("amount:" + "{0:>3f}".format(200*((1+6/100)**n)))
```

```
year: 1 amount:212.000000
year: 2 amount:224.720000
year: 3 amount:238.203200
year: 4 amount:252.495392
year: 5 amount:267.645116
year: 6 amount:283.703822
year: 7 amount:300.726052
year: 8 amount:318.769615
year: 9 amount:337.895792
year:10 amount:358.169539
year:11 amount:379.659712
year:12 amount:402.439294
year:13 amount:426.585652
```

```
year:14 amount:452.180791
year:15 amount:479.311639
year:16 amount:508.070337
year:17 amount:538.554557
year:18 amount:570.867831
year:19 amount:605.119900
year:20 amount:641.427094
```

ENGR010SP23PY01GR03230118